National University of Computer and Emerging Sciences

LOST AND FOUND PORTAL

CS 2005 – Database Systems Project

Group Members

Simroz Sohail 24K-0746

Abeela Khan 24K-0965

Zakia Nadeem 24K-0692

Supervised by

Mr. Ghulam Qadir Burghari

FAST School of Computing

National University of Computer and Emerging Sciences

Karachi, Pakistan

April 2026

1. Introduction (Aim & Motivation)

The University Lost and Found Portal is a web-based information system developed as part of the CS 2005 Database Systems course at FAST NUCES during Spring 2026. The project was initiated in April 2026 and completed within the same month by a team of three students: Zakia Nadeem (24K-0692), Simroz Sohail (24K-0746), and Abeela Khan (24K-0965), under the supervision of their course instructor, Mr. Ghulam Qadir Burghari.

1.1 Aim

The primary aim of this project is to design and implement a fully functional, database-driven web application that addresses a genuine problem faced by university students and staff — the loss of personal belongings on campus. The system bridges the gap between individuals who have lost items and those who have found them, leveraging a structured relational database to facilitate efficient item recovery.

1.2 Motivation

Universities are densely populated environments where items are frequently misplaced, left behind in classrooms, libraries, and common areas, or accidentally exchanged. Existing approaches — such as physical notice boards or informal WhatsApp groups — are disorganized, slow, and lack any structured mechanism for matching lost items with found ones.

The motivation behind this portal stems from a desire to solve this real-world problem through technology. Specifically, the project demonstrates practical application of relational database concepts including JOINs, transactions, stored procedures, views, and parameterized queries in a production-quality web application. It serves as a comprehensive capstone project for the CS 2005 Database Systems course, tying theoretical knowledge with hands-on implementation.

2. Background (Research & Project Selection)

2.1 Research

Prior to project selection, the team conducted research into existing lost-and-found systems and identified key limitations. Physical notice boards lack searchability and real-time updates. Generic social media solutions provide no structure for categorization or status tracking. No dedicated, database-backed solution existed for use at FAST NUCES Karachi.

The team reviewed similar systems deployed at foreign universities and open-source projects on GitHub to understand common feature sets and database schemas. The research confirmed that a role-based, database-driven portal with intelligent matching was the most effective approach.

2.2 Project Selection Rationale

The Lost and Found Portal was selected because it satisfies all requirements of the CS 2005 course project while addressing a genuine, observable problem. The domain is rich enough to require all key SQL constructs — multiple related tables, complex joins, transactions, stored procedures, and views — without being overly specialized or requiring external APIs.

The project was also chosen for its scope appropriateness: it is complex enough to demonstrate database mastery but completable within the time constraints of a single semester project. The three-tier architecture (HTML/CSS/JS frontend, PHP backend, MySQL database) maps cleanly onto topics covered in the course curriculum.

2.3 Technology Selection

Layer	Technology	Justification
Frontend	HTML5, CSS3, Bootstrap 5, JS	Responsive UI with minimal overhead
Backend	PHP 8+ with PDO	Server-side scripting with secure DB access
Database	MySQL 8+	Robust relational DBMS, widely taught
Server	XAMPP / WAMP (localhost)	Easy local deployment for academic use

3. Project Specification

3.1 System Overview

The Lost and Found Portal is a role-based, web-accessible application hosted on a local server. It provides a centralized platform with two distinct user roles: regular users (students and staff) and administrators. All application data is persisted in a MySQL relational database named lost_found_db.

3.2 Functional Requirements

- User registration with email validation and bcrypt password hashing.
- Secure login/logout with session management.
- Authenticated users can report lost items with title, category, description, location, and date.
- Authenticated users can report found items in the same structured format.
- Automatic matching of lost items to found items via stored procedure.
- Users can submit ownership claims on found items with proof text.
- Administrators can approve or reject claims through a transactional operation.
- Administrators can record physical handovers of items to claimants.
- Public landing page displays live statistics and recent reports.

3.3 Non-Functional Requirements

- Security: All database queries use parameterized PDO statements to prevent SQL injection.
- Data integrity: Foreign key constraints and unique constraints enforce relational integrity.
- Reliability: Critical operations (claim approval, handover) are wrapped in database transactions.
- Usability: Bootstrap 5 ensures a responsive design compatible with desktop and mobile browsers.
- Maintainability: Modular PHP includes separate concerns (db.php, auth.php, layout.php).

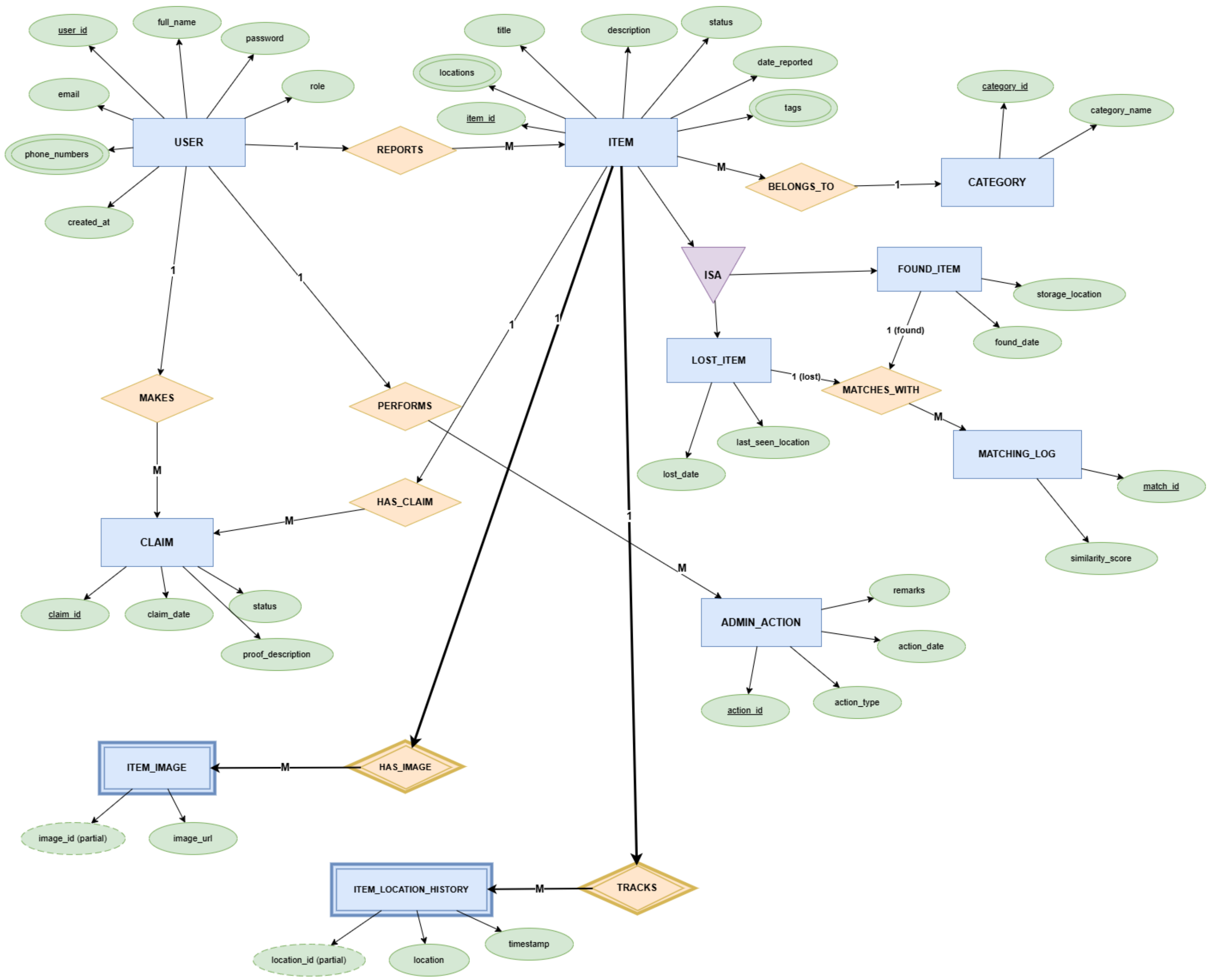
3.4 Database Specification — ERD Description

The database consists of six interrelated tables. The entity-relationship model captures the following associations:

- Users (1) — report — (N) Lost_Items
- Users (1) — report — (N) Found_Items
- Users (1) — submit — (N) Claims
- Found_Items (1) — receive — (N) Claims
- Claims (1) — audited by — (N) Admin_Actions

- Lost_Items (N) — matched with — (N) Found_Items [via Matches bridge table]

ER Diagram:



3.5 Schema Summary

Table	Primary Key	Key Foreign Keys	Purpose
Users	user_id	—	Stores all user accounts
Lost_Items	lost_id	user_id → Users	Records lost item reports
Found_Items	found_id	user_id → Users	Records found item deposits
Claims	claim_id	user_id, found_id	Ownership claims on found items

Table	Primary Key	Key Foreign Keys	Purpose
Admin_Actions	action_id	admin_id → Users, claim_id	Audit log of admin decisions
Matches	match_id	lost_id, found_id	Auto-generated item pairings

4. Problem Analysis

4.1 Problem Statement

FAST NUCES Karachi has no structured digital system for managing lost and found items. Students currently rely on informal channels — WhatsApp groups, physical boards, or word-of-mouth — leading to low recovery rates, no accountability, and no audit trail.

4.2 Identified Problems

- No centralized repository: Lost and found reports are scattered across unrelated platforms.
- No matching mechanism: There is no automated way to connect a lost item report with a found item report of the same type.
- No role-based access: Anyone can claim to own a found item without verification.
- No status tracking: Once an item is reported, there is no lifecycle tracking from discovery to return.
- No admin oversight: There is no mechanism for authorized staff to manage and verify claims.
- No data persistence: Reports made informally are lost when chat histories are cleared or boards are removed.

4.3 Constraints

- The system must be deployable on a local web server (XAMPP/WAMP) without cloud infrastructure.
- The database must use MySQL 8 to comply with course requirements.
- The backend must use PHP 8 with PDO for all database communication.
- The project must be completable within one semester (Spring 2026).

4.4 Feasibility Analysis

Given the availability of free tools (XAMPP, MySQL Workbench, VS Code), the existing team expertise in PHP and SQL from prior coursework, and the well-defined scope, the project is technically and operationally feasible within the given constraints. The three-member team was able to distribute work across database design, backend logic, and frontend implementation.

5. Solution Design (Project Detail, Functionality & Features)

5.1 System Architecture

The application follows a classic three-tier architecture:

- Presentation Layer: HTML5, CSS3, Bootstrap 5, and JavaScript. All pages are rendered server-side by PHP with Bootstrap components for responsive layout.
- Application Layer: PHP 8 using PDO (PHP Data Objects). All business logic, validation, session management, and database communication reside here.
- Data Layer: MySQL 8 database (lost_found_db) containing six tables, three views, and three stored procedures.

5.2 Key Features & Functionality

5.2.1 User Registration & Authentication

New users register with their name, university email, and password. Passwords are hashed using PHP bcrypt (password_hash()). Login is handled via parameterized PDO queries; passwords are verified with password_verify(). Sessions maintain state across pages. Role-based redirects ensure students land on the user dashboard and admins land on the admin panel.

5.2.2 Reporting Lost Items

Authenticated users fill a form with title, category (Electronics, Clothing, Books, ID/Cards, Keys, Bags, Stationery, Other), description, location, and date lost. On submission, a parameterized INSERT adds the record to Lost_Items and immediately calls the stored procedure sp_generate_matches to populate potential matches.

5.2.3 Reporting Found Items

Mirrors the lost item flow. Found items are inserted into Found_Items with status available. When a match exists, users on the lost-item side are notified through their dashboard.

5.2.4 Intelligent Matching System

The stored procedure sp_generate_matches(lost_id) is invoked after every new lost item report. It scans the Found_Items table for items in the same category and computes a match score: 2 points if both category and location match, 1 point for category-only matches. Matches are stored in the Matches table and displayed on the user dashboard.

5.2.5 Claims Management

Users can submit a claim on any available found item by providing proof of ownership. A unique database constraint (user_id, found_id) prevents duplicate claims. Admins review claims in the Manage Claims panel and can approve or reject them. Approval triggers a PDO transaction that atomically updates the claim status, marks the found item as claimed, and logs the action in Admin_Actions.

5.2.6 Admin Control Panel

The admin dashboard displays system-wide statistics using correlated scalar subqueries: total users, lost items, found items, claims, pending claims, recovered items, and recovery rate (using NULLIF to guard against division by zero). A GROUP BY query breaks down found items by category with conditional aggregation using CASE expressions.

5.2.7 Handovers

After a claim is approved, the admin records the physical handover of the item to the claimant. This updates the Found_Items status to handed_over and logs the action in Admin_Actions within a transaction.

5.3 Database Views

View Name	Tables Joined	Purpose
v_active_matches	Matches, Lost_Items, Found_Items	Shows active matched item pairs for user dashboard
v_claim_summary	Claims, Users, Found_Items	Full claim details with claimant identity
v_user_activity	Users, Lost_Items, Found_Items, Claims	Per-user activity statistics

5.4 Stored Procedures

Procedure	Parameters	Description
sp_generate_matches	lost_id	Auto-generates match pairs when a lost item is reported
sp_approve_claim	claim_id, admin_id, remarks	Transactionally approves a claim and updates related records
sp_system_stats	None	Returns two result sets: system totals and category breakdown

6. Implementation & Testing Phaseh

6.1 Development Environment

- IDE: Visual Studio Code
- Web Server: XAMPP (Apache + MySQL)
- Database Tool: phpMyAdmin / MySQL Workbench
- Browser: Google Chrome / Mozilla Firefox

- Version Control: Git

6.2 Implementation Phases

Phase 1 — Database Design & Schema

The team began by designing the ER diagram on paper, identifying all entities, attributes, and relationships. The schema was then implemented in database.sql, which creates all six tables with appropriate data types, constraints, indexes, and foreign keys. Three views and three stored procedures were also defined in this file.

Phase 2 — Backend (PHP + PDO)

The includes/ directory was set up first, establishing the database connection (db.php) and authentication helpers (auth.php). Root-level scripts (register.php, login.php, logout.php) were implemented next, followed by the user/ module and finally the admin/ module. Each script uses only parameterized PDO statements for database interaction.

Phase 3 — Frontend (Bootstrap 5)

layout.php provides shared header and footer HTML. Custom styles are defined in css/style.css using CSS custom properties for theming. The landing page (index.php) uses animated JavaScript counters for statistics and Bootstrap card components for item listings. All forms are styled consistently across pages.

Phase 4 — Integration & Testing

The complete system was tested end-to-end with sample data seeded by database.sql. Test scenarios included user registration with duplicate email detection, login with correct and incorrect credentials, reporting lost and found items, verifying automatic match generation, submitting claims and testing the duplicate-claim constraint, approving and rejecting claims as admin, and recording handovers.

6.3 Testing Results

Test Case	Expected Result	Status
Register with duplicate email	Error: email already exists	Pass
Login with wrong password	Error: invalid credentials	Pass
Report lost item → match generated	Matches table populated	Pass
Submit duplicate claim	Error: unique constraint violated	Pass
Approve claim (transaction)	Claim, Found_Item, Admin_Actions updated atomically	Pass

Test Case	Expected Result	Status
Admin stats calculation	Correct counts and recovery rate	Pass
Search with keyword + category filter	Filtered results returned	Pass
Non-admin access to admin pages	Redirected to user dashboard	Pass

7. Project Breakdown Structure (Workload Distribution & Timeline)

7.1 Team Responsibilities

Team Member	Roll No.	Primary Responsibilities
Simroz Sohail	24K-0746	Database schema design, stored procedures, admin module (manage_claims.php, handovers.php, dashboard.php)
Abeela Khan	24K-0965	User module (dashboard.php, report_lost.php, report_found.php, search.php), frontend styling
Zakia Nadeem	24K-0692	Authentication (login.php, register.php, auth.php), database views, testing & documentation

7.2 Project Timeline

Week	Dates (April 2026)	Tasks Completed
Week 1	Apr 1 – Apr 7	Requirement gathering, ERD design (hand-drawn), project planning, technology selection
Week 2	Apr 8 – Apr 14	Database schema finalized, database.sql written, ERD recreated on MySQL Workbench, sample data seeded
Week 3	Apr 15 – Apr 21	Backend implementation: includes/, register.php, login.php, logout.php, user/ module
Week 4	Apr 22 – Apr 26	Admin module, frontend polish, integration testing, bug fixes, report writing, final submission

7.3 Work Distribution Breakdown

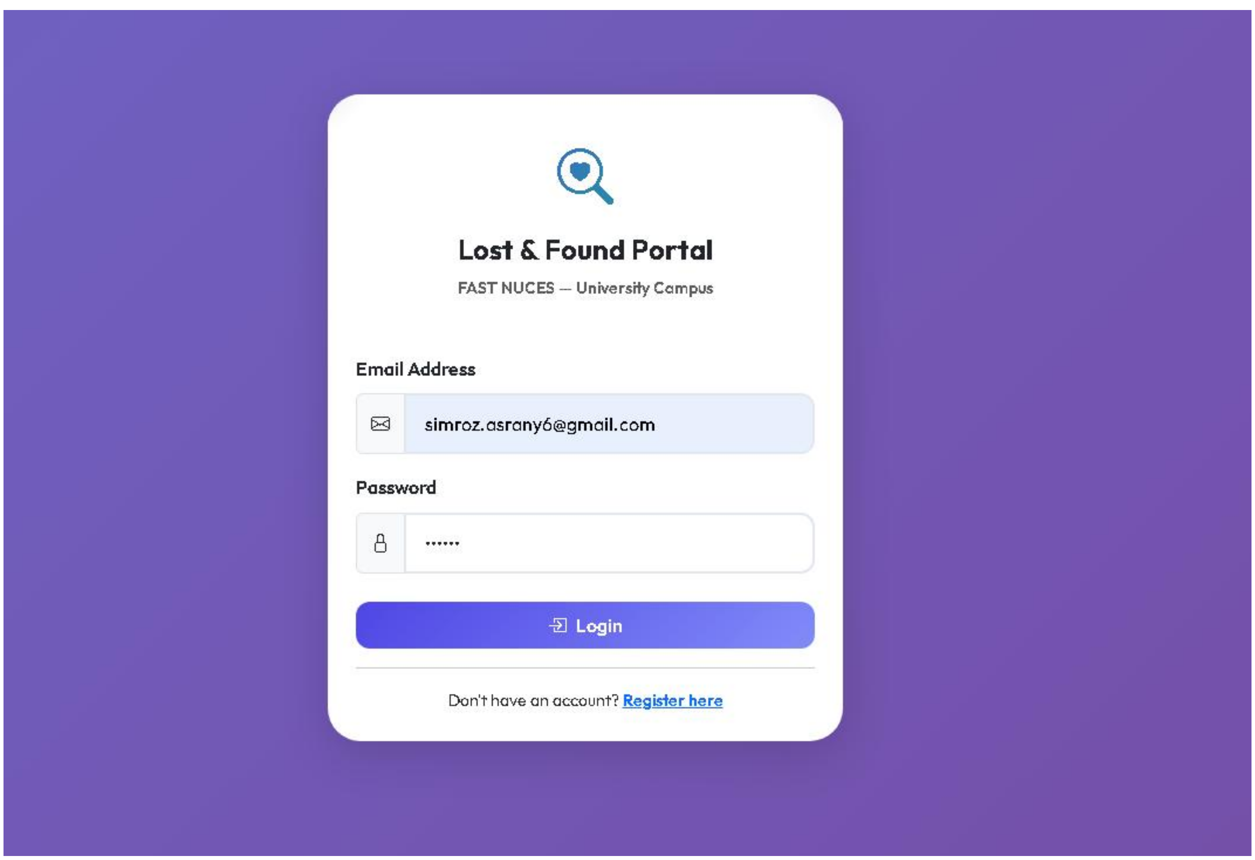
Module / Task	Assignee	Effort (Hours)
ER Diagram (hand-drawn)	All members	3
ER Diagram (MySQL Workbench)	Abeela Khan	2
database.sql (tables + constraints)	Abeela Khan	5

Module / Task	Assignee	Effort (Hours)
Stored procedures	Simroz Sohail	4
Views	Zakia Nadeem	2
includes/ (db.php, auth.php)	Zakia Nadeem	3
layout.php + style.css	Simroz Sohail	4
index.php (landing page)	Simroz Sohail	3
login.php + register.php	Zakia Nadeem	3
user/ module (5 files)	Abeela Khan	6
admin/ module (4 files)	Simroz Sohail	7
Testing & bug fixing	All members	4
Project report writing	All members	5

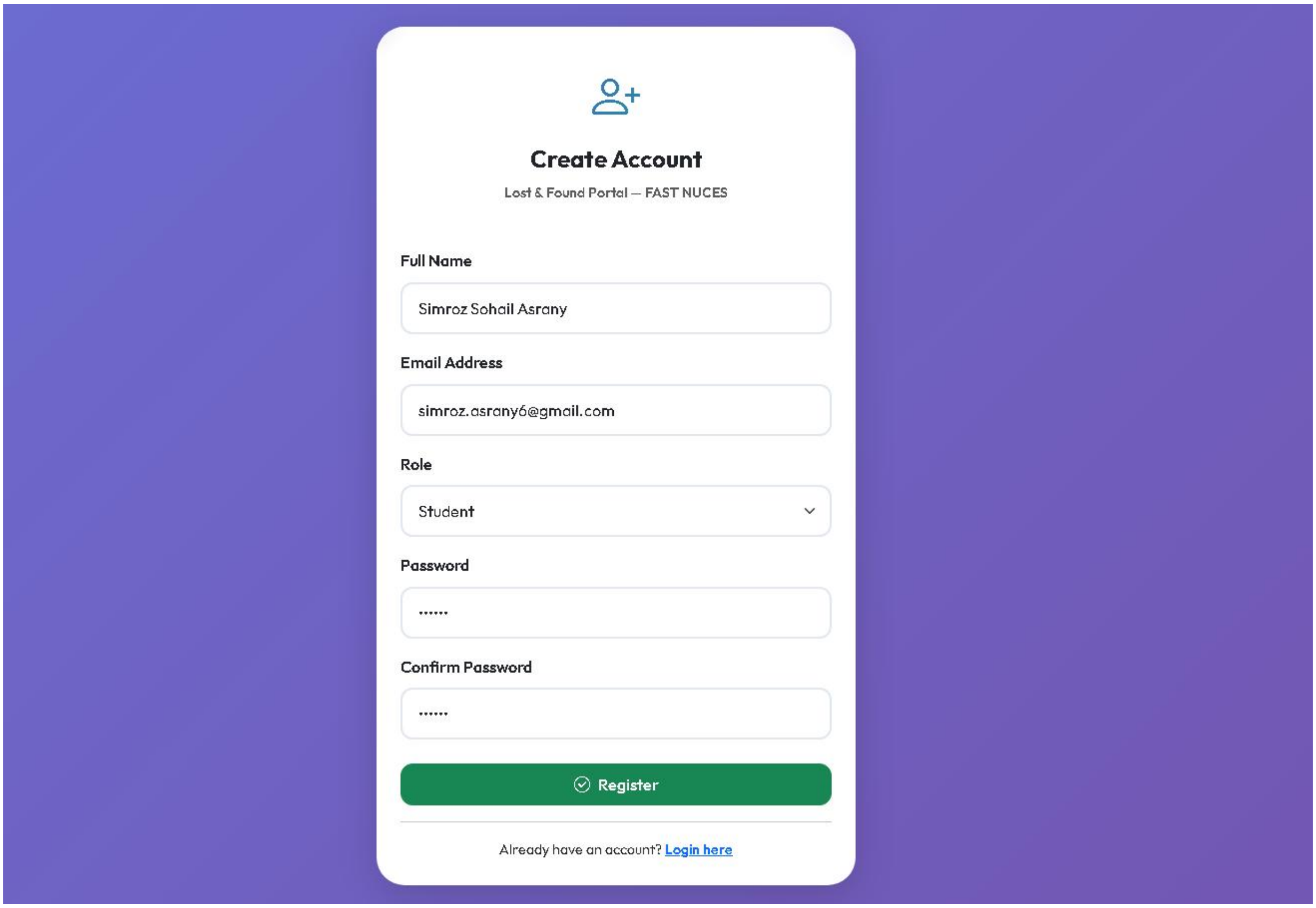
8. Results (Output Screenshots)

The following screenshots illustrate the working output of the Lost and Found Portal at each key stage of the user and administrator workflows.

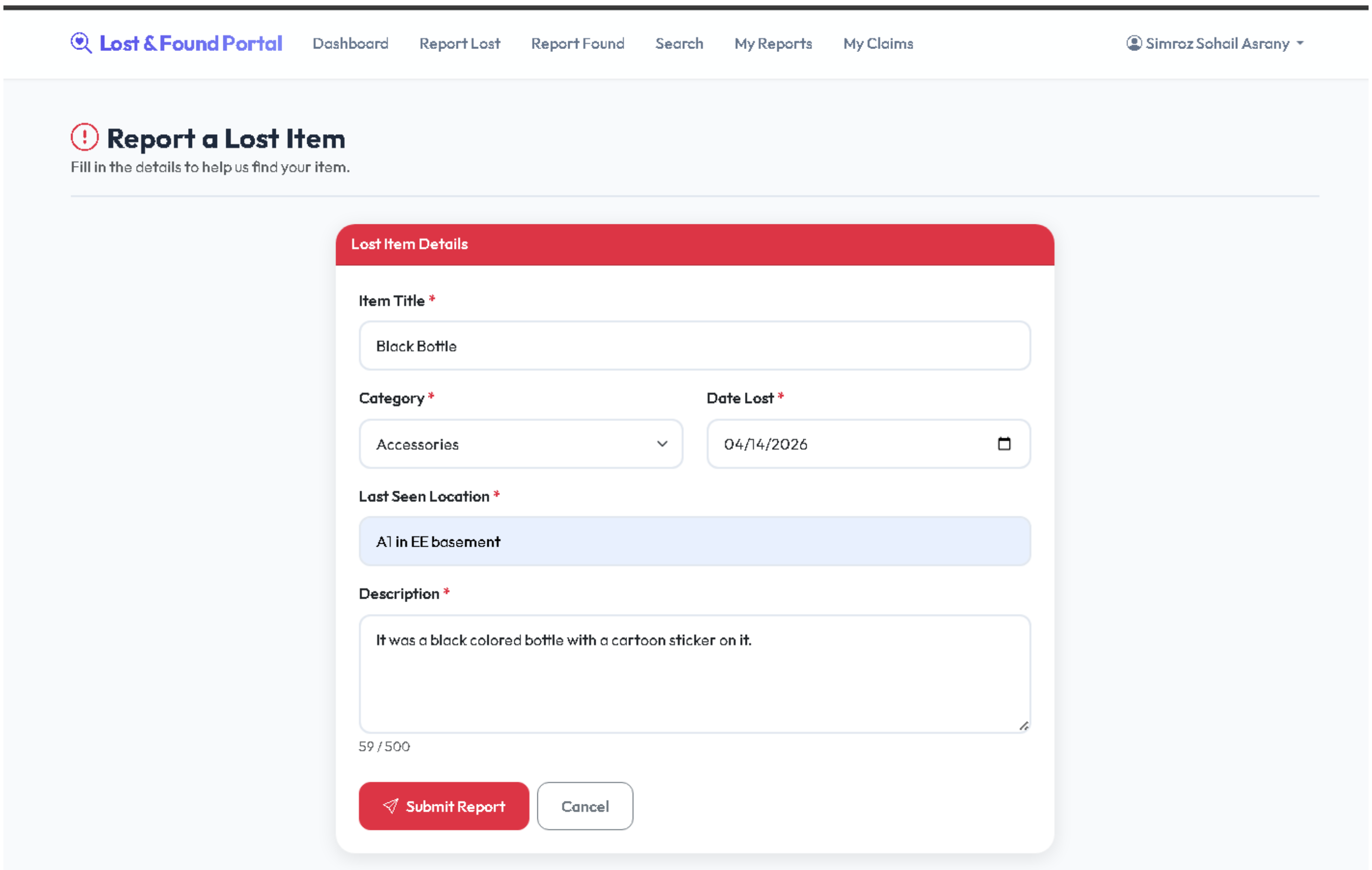
User Login Page:



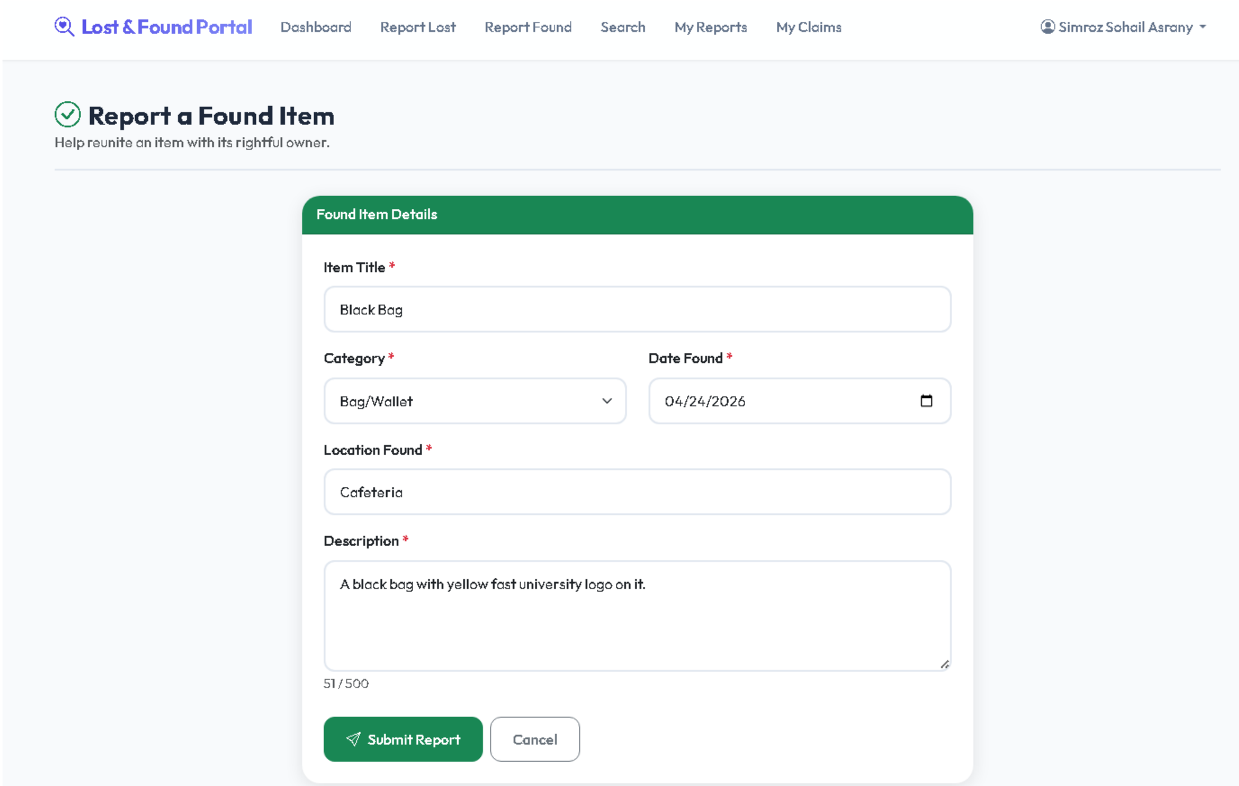
User Registration Page:



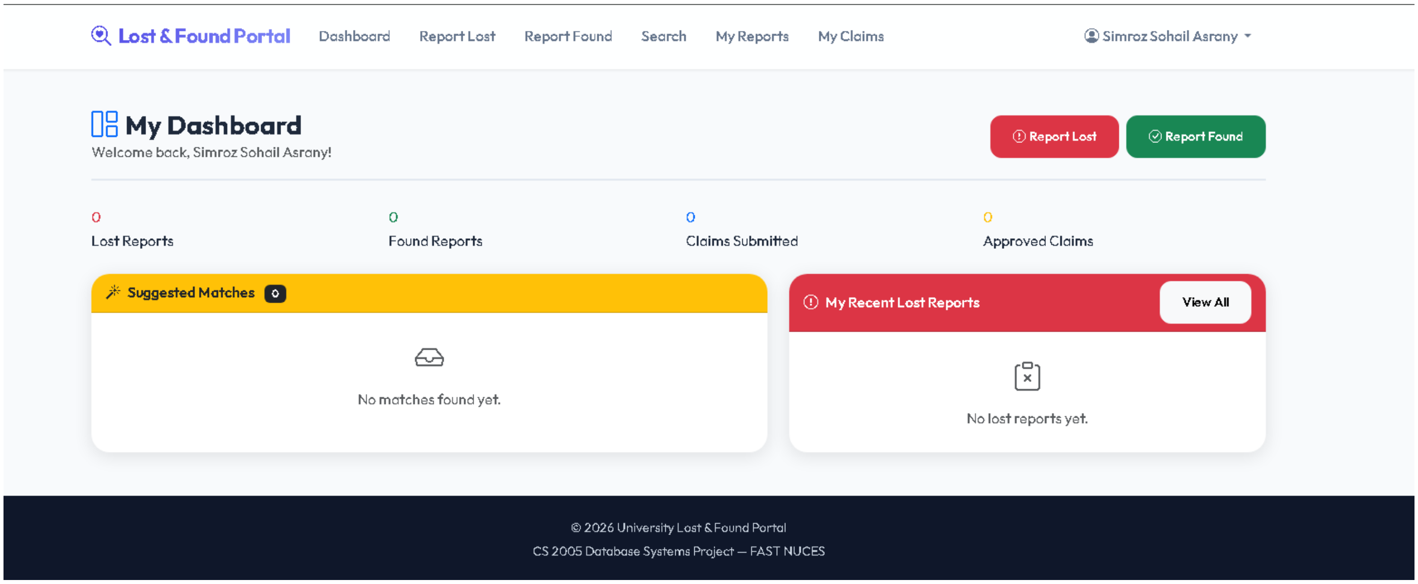
Report lost item:



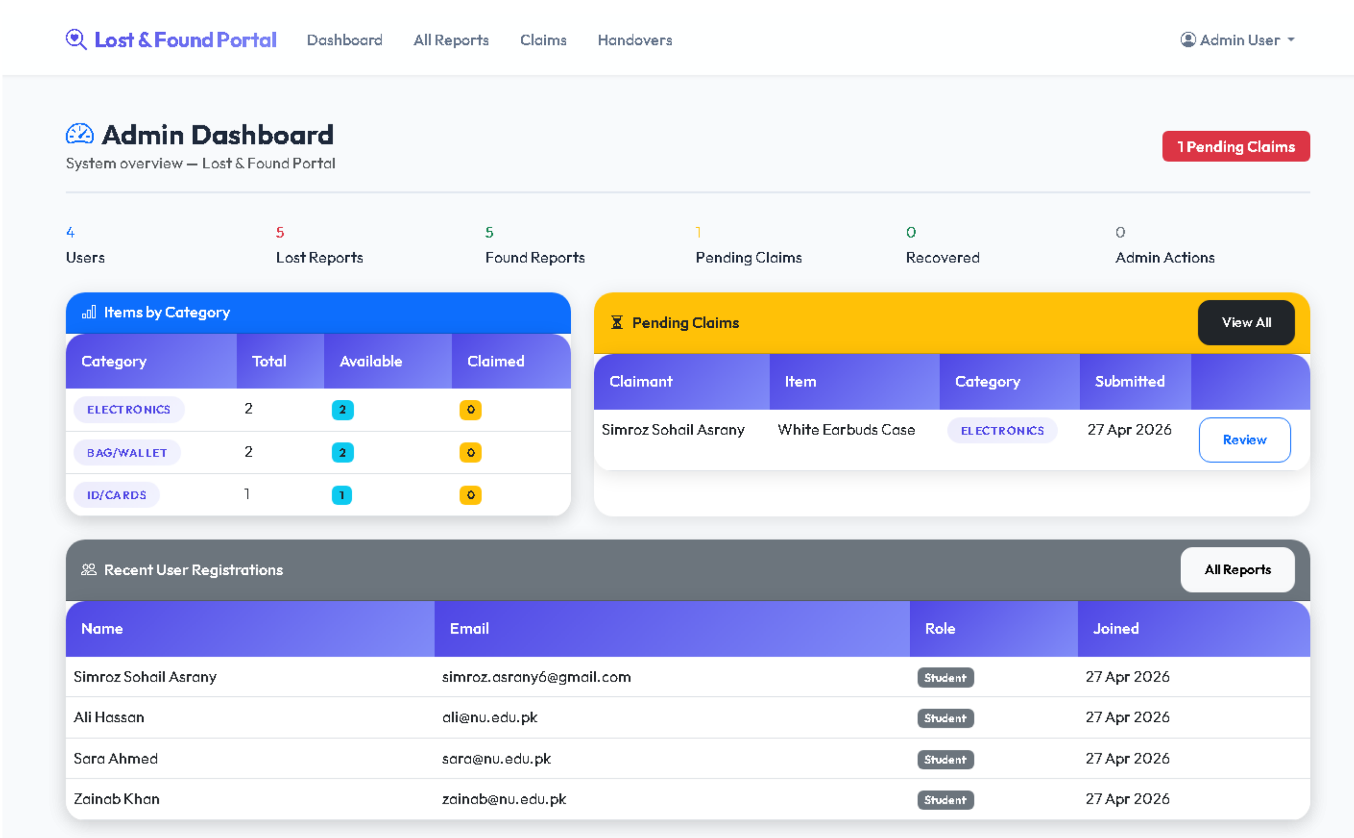
Report found item:



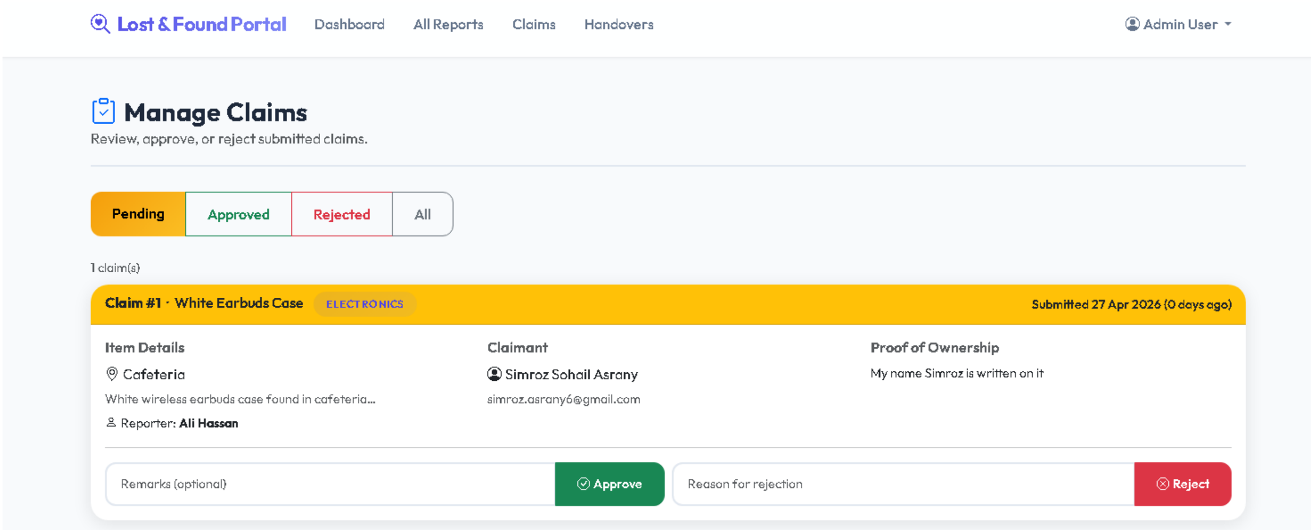
User Dashboard:



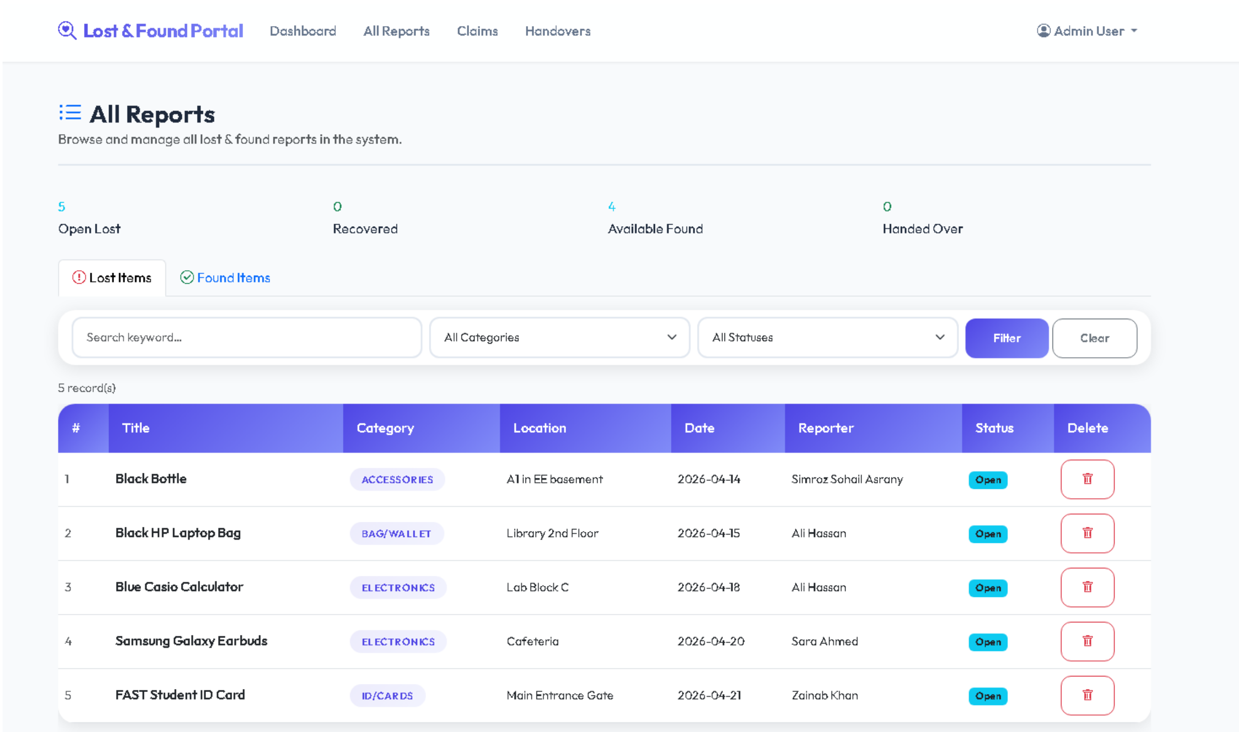
Admin Dashboard:



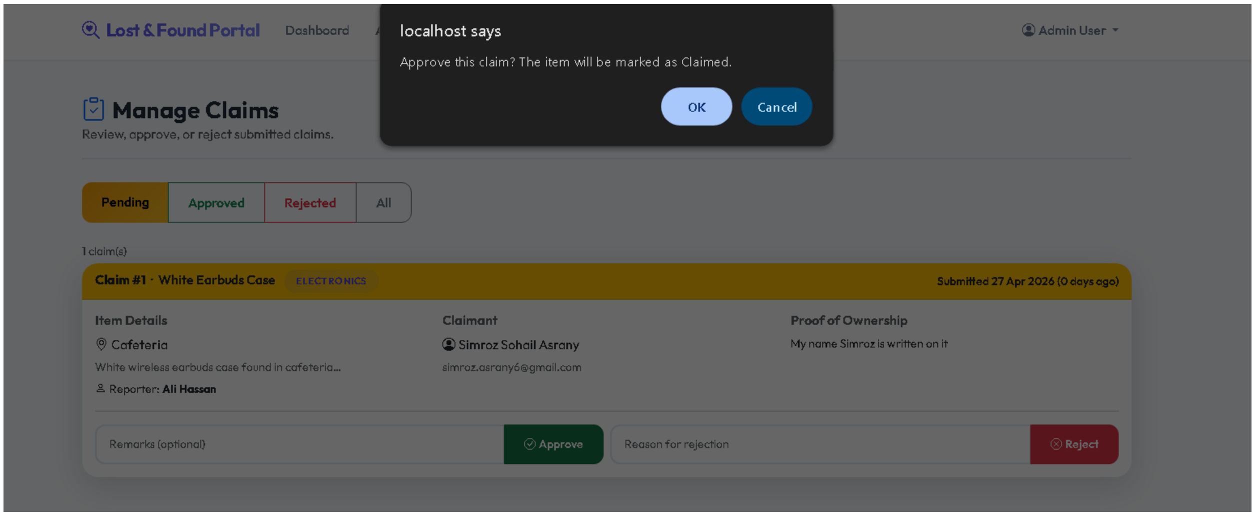
Manage claims page:



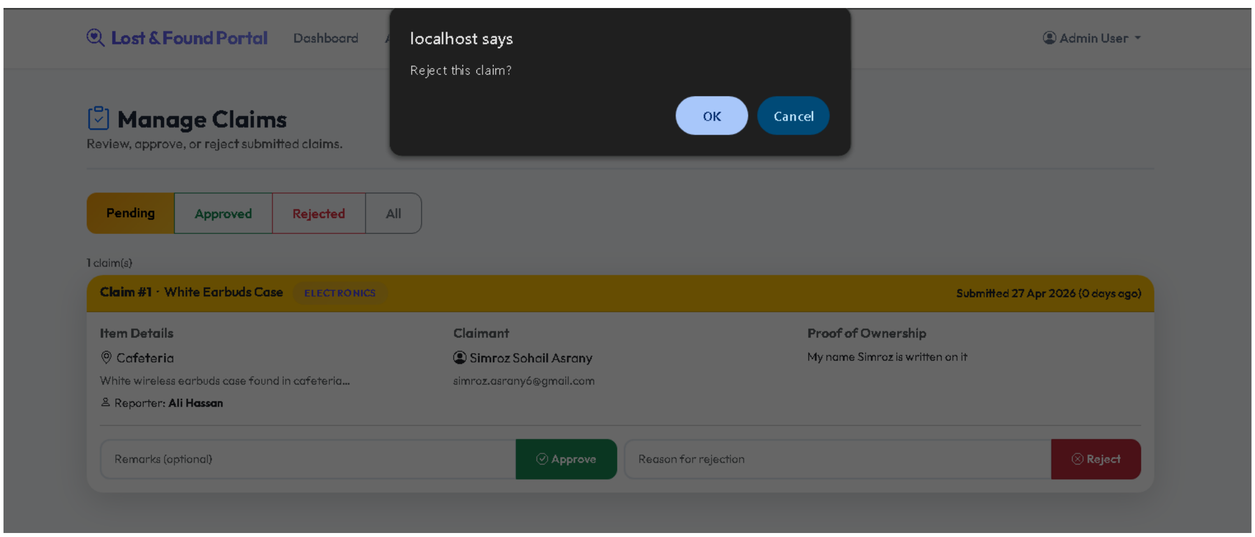
All Reports:



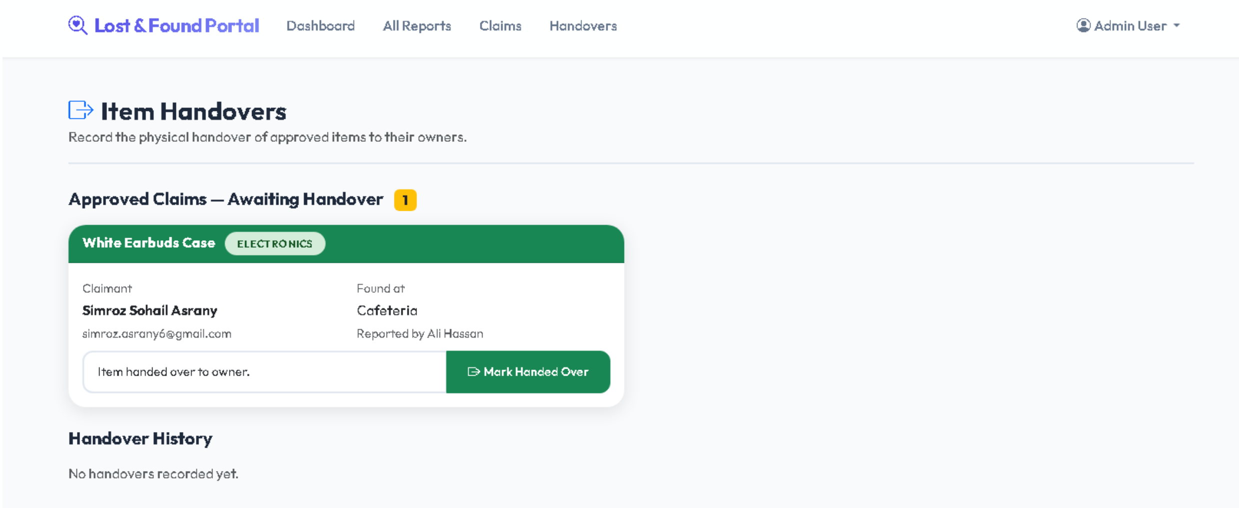
Approving claim:



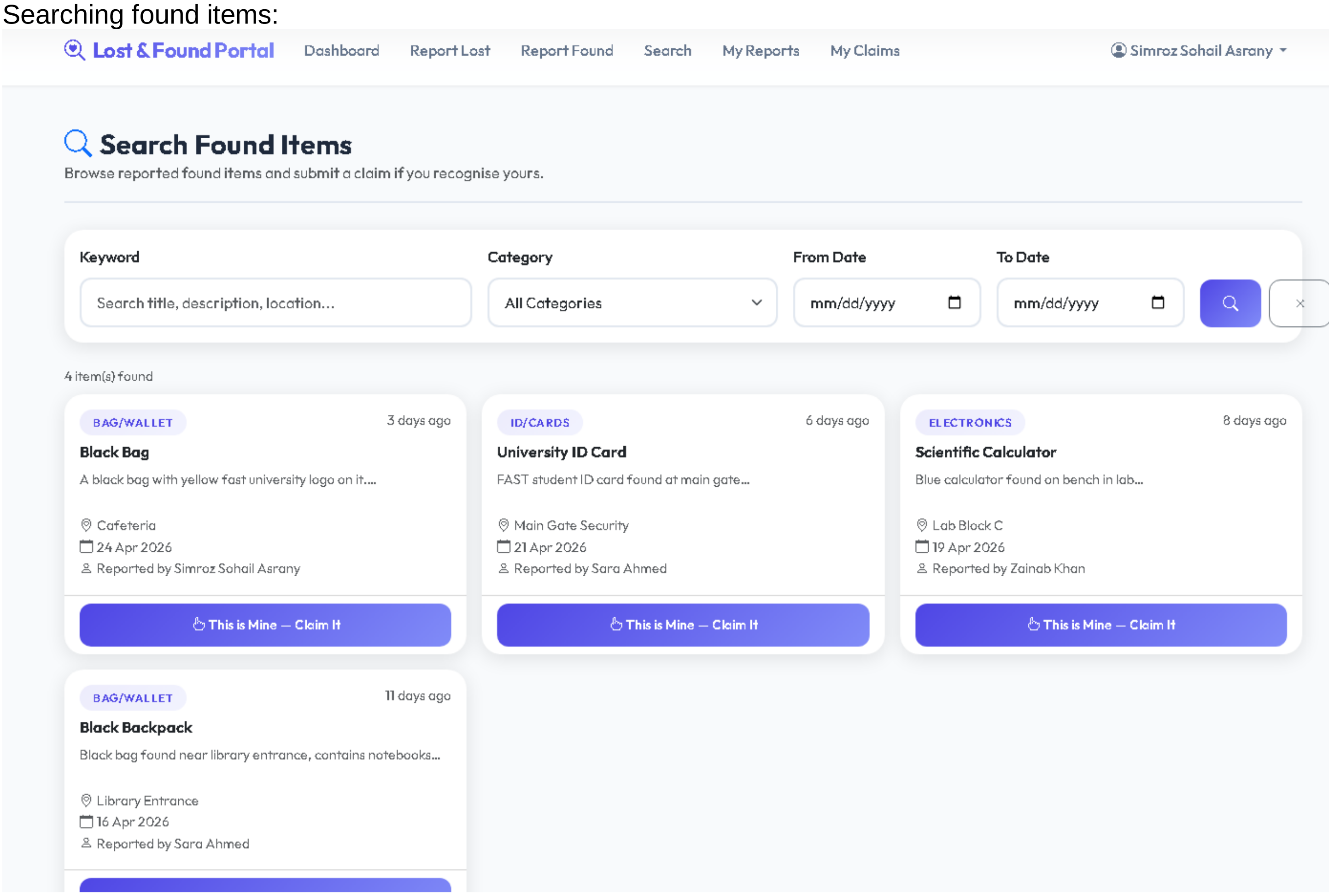
Rejecting claim:



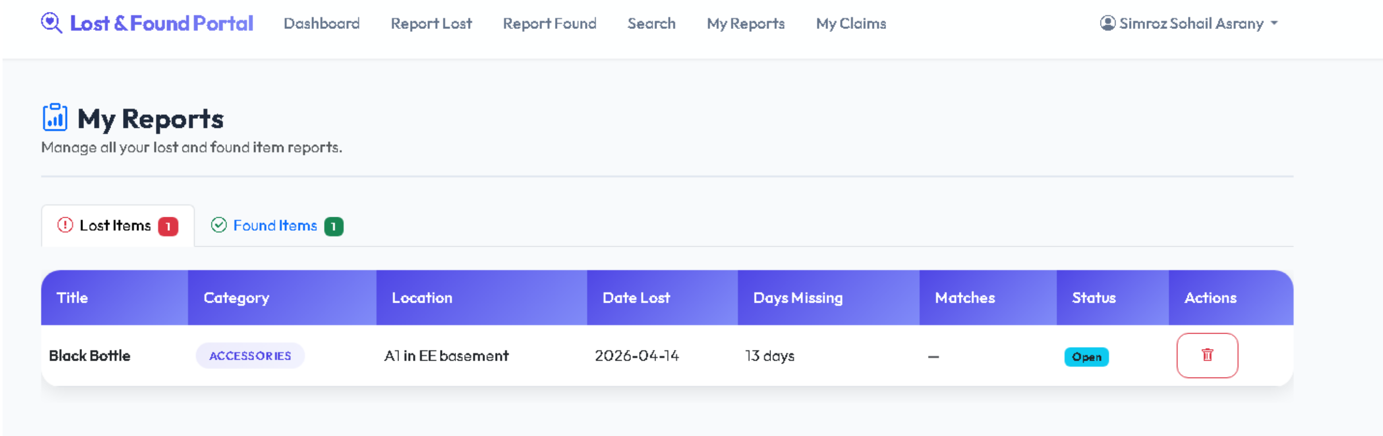
Handovers:



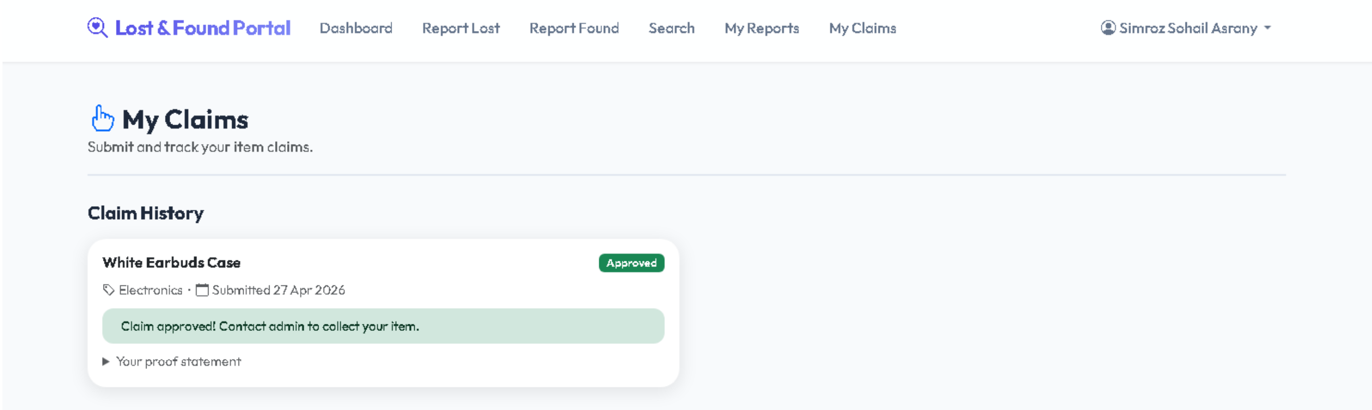
Searching found items:



User reports:



User claims:



9. Conclusion (Summary & Discussion)

9.1 Summary

The University Lost and Found Portal successfully delivers a complete, production-quality web application backed by a well-structured relational database. The system directly addresses the absence of a structured item-recovery mechanism at FAST NUCES Karachi by providing a centralized, role-based digital platform accessible through any web browser.

From a technical standpoint, the project demonstrates comprehensive mastery of the SQL and database concepts prescribed in the CS 2005 curriculum. The schema is normalized, all relationships are enforced with foreign key constraints and unique constraints, and every critical operation is protected with explicit transactional boundaries. Three stored procedures encapsulate key business logic at the database level, while three SQL views simplify complex multi-table queries across the application. The exclusive use of parameterized PDO queries throughout the PHP backend ensures the system is secure against SQL injection.

9.2 SQL Features Demonstrated

SQL Feature	Implementation Location
JOINs (INNER, multi-table)	All dashboard, search, and admin pages
Subqueries (correlated + scalar)	User dashboard stats, my_reports claim count
GROUP BY + Aggregate Functions	Admin dashboard category breakdown
CASE Expressions	Match scoring in sp_generate_matches, category aggregation
Built-in Functions (COUNT, DATEDIFF, DATE_FORMAT, NULLIF, LOWER)	Throughout application
Parameterized Queries (PDO)	All form submissions, login, search
INSERT / UPDATE / DELETE	Registration, item reporting, claim handling, deletions
TRANSACTIONS	manage_claims.php, handovers.php, sp_approve_claim
Views (3)	v_active_matches, v_claim_summary, v_user_activity
Stored Procedures (3)	sp_generate_matches, sp_approve_claim, sp_system_stats
Indexes	On email, role, category, status columns for query optimization
Unique Constraints	User email; (user_id, found_id) in Claims table

9.3 Learning Outcomes

The project reinforced several important concepts beyond the database curriculum. Role-based access control, session management, and password hashing provided exposure to web application security fundamentals. The modular code structure — separating database connection, authentication, and layout into reusable includes — demonstrated good software engineering practice. The iterative nature of the project (designing the ER diagram first, then implementing the schema, then building the application on top) reinforced the importance of database-first design in web development.

9.4 Limitations & Future Work

The current implementation operates on a local server and would require cloud deployment (e.g., cPanel or a VPS with Apache/MySQL) for institution-wide use. Future enhancements could include:

- Email notification system to alert users when a match is found or a claim is updated.
- Image upload support for lost and found item reports to improve identification accuracy.
- Advanced full-text search using MySQL FULLTEXT indexes for better keyword matching.
- Mobile application frontend to increase accessibility for students on campus.

- Integration with the university's existing student information system for automatic role assignment.

9.5 Closing Remarks

The University Lost and Found Portal stands as a complete demonstration of how relational database theory translates into practical, real-world software. The project not only fulfills the academic requirements but also produces a genuinely useful tool that, if deployed, would benefit the entire FAST NUCES campus community.